Sistemas de Numeração

Jogos Digitais

Sistemas de Numeração

- Objetivo: Compreender os sistemas de numeração utilizados em sistemas computacionais
- Bases e sistemas:
  - A contagem sempre foi importante na existência humana
  - Foram criadas diversas formas de contagem e representação de valores
  - Base é o valor de referência para criação da representação numérica
  - Sistemas comuns: decimal, binário, octal, hexadecimal, ...

Sistemas de Numeração

- Notação Posicional:
  - Numerais hindu-arábicos
  - Operações podem ser realizadas posicionando números sobre os outros

$\mathbf{R} = \mathbf{d}_{n-1} * \mathbf{b}^{n-1} + \mathbf{d}_{n-2} * \mathbf{b}^{n-2} + \ldots + \mathbf{d}_1 * \mathbf{b}^1 + \mathbf{d}_0 * \mathbf{b}^0$, sendo **d** o algarismo e **b** a base

$\mathbf{135} = \mathbf{1} * \mathbf{10}^2 + \mathbf{3} * \mathbf{10}^1 + \mathbf{5} * \mathbf{10}^0$

Decimal (10)	Binário (2)	Hexadecimal (16)	Octal (8)
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	8	10
9	1001	9	11
10	1010	A	12
11	1011	B	13
12	1100	C	14
13	1101	D	15
14	1110	E	16
15	1111	F	17

Sistemas de Numeração

- Conversão de **binário para decimal**:

$1011_2 = ?_{10}$

base = 2, dígitos = 4

$R = d_{n-1} * b^{n-1} + d_{n-2} * b^{n-2} + ... + d_1 * b^1 + d_0 * b^0$, sendo **d** o dígito e **b** a base

$R = 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 11_{10}$

Sistemas de Numeração

- Conversão de **hexadecimal para decimal**:

$AB_{16} = ?_{10}$

base = 16, dígitos = 2

$R = d_{n-1} * b^{n-1} + d_{n-2} * b^{n-2} + ... + d_1 * b^1 + d_0 * b^0$, sendo **d** o dígito e **b** a base

$R = 10 * 16^1 + 11 * 16^0 = 171_{10}$

Sistemas de Numeração

- Conversão de **octal para decimal**:

$25_8 = ?_{10}$

base = 8, dígitos = 2

$R = d_{n-1} * b^{n-1} + d_{n-2} * b^{n-2} + ... + d_1 * b^1 + d_0 * b^0$, sendo **d** o dígito e **b** a base

$R = 2 * 8^1 + 5 * 8^0 = 21_{10}$

Sistemas de Numeração

- Conversão de **decimal para binário**:

11_{10} = $?_2$

Utilizar divisões sucessivas do número decimal pela base desejada até o resultado ser menor ou igual à base. O último resultado é o maior algarismo.

11 / 2 = 5 resto **1**

5 / 2 = 2 resto **1**

2 / 2 = **1** resto **0** **=> 1011_2**

Sistemas de Numeração

- Conversão de **decimal para hexadecimal**:

2754_{10} = $?_{16}$

Utilizar divisões sucessivas do número decimal pela base desejada até o resultado ser menor ou igual à base. O último resultado é o maior algarismo.

2754 / 16 = 172 resto **2**

172 / 16 = **10** resto **12 => AC 2_{16}**

Sistemas de Numeração

- Conversão de **decimal para octal**:

3964_{10} = $?_8$

Utilizar divisões sucessivas do número decimal pela base desejada até o resultado ser menor ou igual à base. O último resultado é o maior algarismo.

3964 / 8 = 495 resto **4**

495 / 8 = 61 resto **7**

61 / 8 = **7** resto **5 => 7574_8**

Sistemas de Numeração

- **Soma** de números binários (Método padrão):

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 e vai um

```
  1101
+ 1101
-------
```

Sistemas de Numeração

- **Soma** de números binários (Método padrão):

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 e vai um

```
   1
 1101
+1101
-------
    0
```

Sistemas de Numeração

- **Soma** de números binários (Método padrão):

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 e vai um

```
     1
   1101
+  1101
-------
     10
```

Sistemas de Numeração

- **Soma** de números binários (Método padrão):

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 e vai um

```
  1   1
  1101
+ 1101
-------
   010
```

Sistemas de Numeração

- **Soma** de números binários (Método padrão):

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 e vai um

```
1  1   1
   1101
+  1101
   -------
    1010
```

Sistemas de Numeração

- **Soma** de números binários (Método padrão):

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 e vai um

```
 1  1   1
   1101
+  1101
   -------
  11010
```

Sistemas de Numeração

- **Subtração** de números binários (Método padrão):

0 - 0 = 0

0 - 1 = 1 pega 2 emprestado

1 - 0 = 1

1 - 1 = 0

```
  1100
- 1001
-------
```

Sistemas de Numeração

- **Subtração** de números binários (Método padrão):

0 - 0 = 0

0 - 1 = 1 pega 2 emprestado

1 - 0 = 1

1 - 1 = 0

```
   2
  1000
- 1001
-------
```

Sistemas de Numeração

- **Subtração** de números binários (Método padrão):

0 - 0 = 0

0 - 1 = 1 pega 2 emprestado

1 - 0 = 1

1 - 1 = 0

```
     1 2
   1000
-  1001
-------
      1
```

Sistemas de Numeração

- **Subtração** de números binários (Método padrão):

0 - 0 = 0

0 - 1 = 1 pega 2 emprestado

1 - 0 = 1

1 - 1 = 0

```
     1 2
  1000
- 1001
-------
    11
```

Sistemas de Numeração

- **Subtração** de números binários (Método padrão):

0 - 0 = 0

0 - 1 = 1 pega 2 emprestado

1 - 0 = 1

1 - 1 = 0

```
     1 2
   1000
-  1001
-------
    011
```

Sistemas de Numeração

- **Subtração** de números binários (Método padrão):

0 - 0 = 0

0 - 1 = 1 pega 2 emprestado

1 - 0 = 1

1 - 1 = 0

```
     1 2
   1000
-  1001
-------
   0011
```

Sistemas de Numeração

- Números **octais** e **hexadecimais** (Método padrão):
  - Operações em outras bases é similar
  - Na **soma**, quando a base é extrapolada, usa-se o **vai um**
  - Na **subtração**, quando o subtraendo é maior que o minuendo, **pega-se emprestado** um dígito adiante, que tem **o valor da base**

	68		716
71	~~7~~1	8A	~~8~~A
- 17	- 17	- 1B	- 1B
---	---	---	---
	52		6F

Sistemas de Numeração

- **Sinal** em binários (bit reservado):

BIN	DEC
00	0
01	1
10	2
11	3

Bit de sinal (signed)

BIN	DEC
000	0
001	1
010	2
011	3
100	- 0
101	- 1
110	- 2
111	- 3

- Duas representações para o número 0

Sistemas de Numeração

- **Subtração** de binários (Método de Von Neumann):
  - Complemento de 1
  - Complemento de 2
  - Não utiliza instrução de subtração, apenas soma
  - Maior eficiência para cálculos binários na ULA
  - Registrador de N bits

Sistemas de Numeração

- **Subtração** de binários (Método de Von Neumann):

DEC	Complemento de 1
0	000
1	001
2	010
3	011
- 0	**111**
- 1	**110**
- 2	**101**
- 3	**100**

- Duas representações para o número 0

- Desperdício de espaço no registrador

- Registrador de 3 bits (inteiro)

- **Complemento de 1: Completar os bits e inverter positivos para obter os números negativos**

Sistemas de Numeração

- **Subtração** de binários (Método de Von Neumann):

DEC	Complemento de 2
0	000
1	001
2	010
3	011
- 4	**100**
- 1	**110 + 1 = 111**
- 2	**101 + 1 = 110**
- 3	**100 + 1 = 101**

- Uma representações para o número 0

- Ganha mais uma representação (-4 decimal)

- Registrador de 3 bits (inteiro)

- **Complemento de 2: Somar 1 bit no complemento de 1**

Sistemas de Numeração

- **Subtração** de binários (Método de Von Neumann):

Calcular 10 – 7 em binário

$10_{10} = 1010_2$

$7_{10} = 111_2$

Complemento de 1 para o número – 7:

Completar o número de dígitos para igualar: **0111** (7)

Inverter: **1000** (-7 no complemento de 1)

Sistemas de Numeração

- **Subtração** de binários (Método de Von Neumann):

Calcular 10 – 7 em binário

$10_{10} = 1010_2$

$7_{10} = 111_2$

Complemento de 2 para o número – 7:

Somar 1 bit no complemento de 1: **1000 + 1** = **1001** (complemento de 2)

Sistemas de Numeração

- **Subtração** de binários (Método de Von Neumann):

Calcular 10 – 7 em binário

$10_{10} = 1010_2$

$7_{10} = 111_2$

$-7_{10} = 1001_2$ (complemento de 2)

Somar 1010 com 1001

$$\begin{array}{r} \cancel{1} \\ 1010 \\ +\ 1001 \\ \hline 0011 = (3_{10}) \end{array}$$

Possíveis representações de sinal

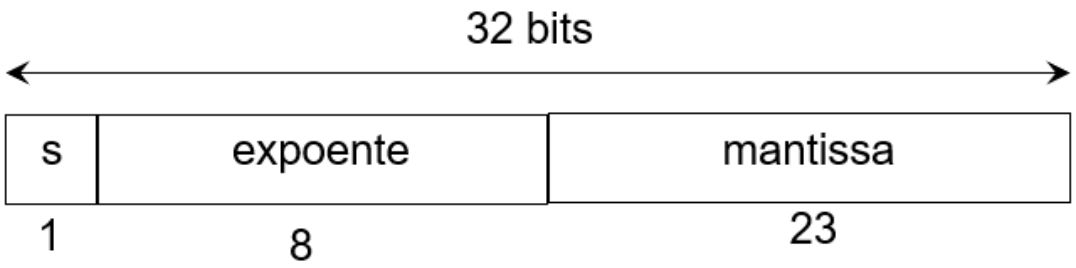
- Sinal Magnitude:
  000 = +0
  001 = +1
  010 = +2
  011 = +3
  100 = -0
  101 = -1
  110 = -2
  111 = -3

Complemento de 1
000 = +0
001 = +1
010 = +2
011 = +3
100 = -3
101 = -2
110 = -1
111 = -0

Complemento de 2
000 = +0
001 = +1
010 = +2
011 = +3
100 = -4
101 = -3
110 = -2
111 = -1

Representação em Ponto Flutuante

$$N = (-1)^S \times F \times 2^E$$



- S – sinal do número
- F – mantissa, normalizada
- E – expoente
- Tamanho: $-2{,}0 \times 10^{38}$ a $2{,}0 \times 10^{38}$.

Representar o número $-0{,}75_{10}$ em ponto flutuante e precisão simples.

- 1ª fase: converter para binário:
  - $0{,}75 \times 2 = 1{,}5 \rightarrow 1$
  - $0{,}5 \times 2 = 1{,}0 \rightarrow 1$
  - $-0{,}75_{10} = -0{,}11_2 = -0{,}11 \times 2^0 = -1{,}1 \times 2^{-1}$
- Em precisão simples:

$$N = (-1)^S \times (1 + Mantissa) \times 2^{(E-Peso)}$$

- Resultado:

$$(-1)^1 \times (1 + 0{,}1000\,0000\,0000\,0000\,0000\,000_{dois}) \times 2^{(126-127)}$$

- Representação:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

1 bit — 8 bits — 23 bits

IEEE-754 Exemplos

	Value	Bit Pattern		
		Sign	Exponent	Fraction
(a)	$+1.101 \times 2^5$	0	1000 0100	101 0000 0000 0000 0000 0000
(b)	-1.01011×2^{-126}	1	0000 0001	010 1100 0000 0000 0000 0000
(c)	$+1.0 \times 2^{127}$	0	1111 1110	000 0000 0000 0000 0000 0000
(d)	$+0$	0	0000 0000	000 0000 0000 0000 0000 0000
(e)	-0	1	0000 0000	000 0000 0000 0000 0000 0000
(f)	$+\infty$	0	1111 1111	000 0000 0000 0000 0000 0000
(g)	$+2^{-128}$	0	0000 0000	010 0000 0000 0000 0000 0000
(h)	+NaN	0	1111 1111	011 0111 0000 0000 0000 0000
(i)	$+2^{-128}$	0	011 0111 1111	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000